

课程笔记

结构化输出与工具调用：LangChain / OpenAI API

五道口纳什 & Claude

2026 年 3 月 28 日

结构化输出/工具调用 LangChain OpenAI API Http Request

视频作者/频道：五道口纳什

发布日期：2025

视频时长：约 9 分钟

视频链接：<https://www.bilibili.com/video/BV1uiUnBFED4>

目录

1 结构化输出与工具调用的本质	2
1.1 为什么需要结构化输出	2
1.2 本章小结	2
2 OpenAI API 层面的实现	2
2.1 Structured Output: response_format 字段	2
2.2 Function Calling: tools 字段	2
2.3 本章小结	3
3 LangChain 层面的封装	3
3.1 with_structured_output	3
3.2 本章小结	3
4 实用技巧：结构化输出实现 CoT	4
5 总结与延伸	4

1 结构化输出与工具调用的本质

1.1 为什么需要结构化输出

在构建复杂 Agent Workflow 时，模块之间需要明确的**接口约定**——一个环节的输出是另一个环节的输入。结构化输出解决了三个核心问题：

1. 不需要在 Prompt 层面约定 Output Format
2. 不需要手动解析复杂的 JSON
3. Function / Tools 信息也不需要显式放入 Prompt

结构化输出

结构化输出某种意义上实现了一种弱版的 **Specification Driven Development**——通过定义 Module 之间的输入输出格式，极大简化了解析和适配工作。其底层原因在于现代大语言模型在结构化输出方面已经训练得非常好。

1.2 本章小结

结构化输出和工具调用是 Agent 开发的基础设施——通过 API 层面的格式约定，省略了大量手工的 Prompt 设计和输出解析工作。

2 OpenAI API 层面的实现

2.1 Structured Output: response_format 字段

通过 `response_format` 字段传入 Pydantic Model 的 JSON Schema：

- 定义 Pydantic Model（如包含 `title` 和 `year` 字段）
- 通过 `model.json_schema()` 将其序列化为 JSON Schema
- 放入 HTTP 请求的 `response_format` 字段
- 模型输出自动符合该 Schema，可直接反序列化为对象

2.2 Function Calling: tools 字段

通过 `tools` 字段注入工具信息：

- 使用 LangChain 的 `@tool` 装饰器注册 Python 函数
- 通过 `convert_to_openai_tool()` 转换为 OpenAI API 要求的格式
- 放入 HTTP 请求的 `tools` 字段
- 模型返回中包含 `function_call` 的实例化内容

HTTP 抓包分析

通过 HTTP 抓包可以清晰看到：

- **Structured Output** 走 `response_format` 字段
- **Function Calling / Bound Tools** 走 `tools` 字段

两者在 HTTP 请求层面是不同的字段，但目的都是约定输入输出格式。

2.3 本章小结

在 OpenAI API 的 HTTP 请求层面，结构化输出通过 `response_format` 实现，工具调用通过 `tools` 实现——理解底层机制有助于 debug 和优化。

3 LangChain 层面的封装

3.1 with_structured_output

LangChain 的 `with_structured_output` 方法进一步简化了使用：

- 直接传入 Pydantic Model
- 底层自动转换为 JSON Schema 并放入 `response_format`
- 返回的结果直接是实例化的对象（比 OpenAI API 多了一步自动反序列化）

两种路径的区别

- `with_structured_output` → 走 `response_format`（JSON Schema 对齐到字段语义）
- `bind_tools` → 走 `tools`（Function Calling 语义）

Pydantic Model 本质上也是一种 Function（实例化类 ≈ 调用构造函数），所以 LangChain 的 `bind_tools` 也可以接受 Pydantic Model。

3.2 本章小结

LangChain 在 OpenAI API 的基础上做了进一步封装——自动序列化、自动反序列化，使用体验更简洁。

4 实用技巧：结构化输出实现 CoT

通过结构化输出实现 Chain of Thought

可以将思考过程封装为一个 Pydantic 结构化输出对象，例如：

- `steps: [Step(explanation=..., output=...)]`
- 每个 Step 包含解释和输出

通过这种方式约定实现 CoT 效果，自然可以比较大地提升模型表现。这是一个非常实用的语言模型使用技巧。

5 总结与延伸

1. **结构化输出和工具调用是 Agent 开发的基础设施**：它们在 API 层面实现了模块间的接口约定。
2. **理解底层 HTTP 请求有助于 debug**：`response_format` vs. `tools` 是两个不同的字段。
3. **LangChain 提供了更高层的封装**：自动序列化/反序列化，但底层仍然是 OpenAI API。
4. **结构化输出可以实现 CoT**：将思考过程结构化，是一个实用的性能提升技巧。
5. **现代 LLM 的结构化输出能力已经很强**：可以省略大量手工的 Prompt 格式约定和输出解析。